



C# - Overview

동명대학교 게임그래픽학과

강영민

개요

▶ C#

- ▶ 현대적이며 범용의 프로그래밍 언어
- ▶ 개발사: 마이크로소프트
- ▶ 유럽 컴퓨터 제 조사 협회(ECMA)와 국제 표준 기구(ISO)의 승인을 받음

▶ 개발자

- ▶ Anders Hejlsberg와 동료들
- ▶ .Net 프레임워크를 개발하는 과정에 C#을 개발하게 됨

▶ 기본 구조

- ▶ CLI(공용 언어 기반, Common Language Infrastructure)을 위해 설계됨
 - ▶ CLI: 여러 종류의 컴퓨터 플랫폼과 구조에서 다양한 고급 언어를 사용할 수 있도록 하는 실행 코드와 런타임(runtime) 환경으로 구성 (Command Line Interface와 혼동 주의)



특징

- ▶ 현대적 프로그래밍 언어의 특성을 갖고 있으며 범용이다
- ▶ 객체 지향이다
- ▶ 컴포넌트 지향이다
- ▶ 배우기 쉽다
- ▶ 구조적이다
- ▶ 효율적인 프로그램을 생산한다
- ▶ 다양한 컴퓨터 플랫폼에서 컴파일할 수 있다
- ▶ .Net 프레임워크의 일부이다

주요 기능

- ▶ Bool 조건
- ▶ 자동 가비지 컬렉션(garbage collection)
- ▶ 표준 라이브러리
- ▶ 어셈블리 버전 관리(assembly versioning)
- ▶ 프로퍼티와 이벤트
- ▶ 델리게이트와 이벤트 관리
- ▶ 사용하기 편리한 제너릭
- ▶ 인덱서(indexer)
- ▶ 조건 컴파일
- ▶ 간단한 멀티쓰레딩
- ▶ LINQ와 람다 표현식
- ▶ 윈도우 운영체제와의 통합성 → .Net5 이상에서는 다양한 운영체제 지원

프로그래밍 구조

- ▶ C# 프로그램은 다음과 같은 요소를 가짐
 - ▶ 네임스페이스(name space) 선언
 - ▶ 클래스
 - ▶ 클래스 메소드
 - ▶ 클래스 속성
 - ▶ Main 메소드
 - ▶ 명령문과 표현
 - ▶ 코멘트

프로그램 예

▶ Hello

```
using System;
namespace HelloWorldApplication {
    class HelloWorld {
        static void Main(string[] args) { /* my first program in C# */
            Console.WriteLine("Hello World");
            Console.ReadKey();
        }
    }
}
```

코드 내용 살펴 보기

- ▶ using System
 - ▶ System 네임스페이스를 포함.
 - ▶ 일반적으로 하나의 프로그램은 여러 개의 using 명령을 가진다.
- ▶ namespace 선언
 - ▶ 네임스페이스는 클래스의 집합 (자바의 package)
- ▶ 클래스 선언
 - ▶ 하나의 메소드(Main)를 가진 클래스
 - ▶ Main 메소드는 static

기본적인 구문

- ▶ C#은 객체지향 언어
 - ▶ 상호작용하는 객체로 프로그램을 구성
 - ▶ 클래스
- ▶ using 키워드
 - ▶ 사용하는 네임스페이스를 설정
- ▶ 클래스 키워드 – 다른 언어와 유사
- ▶ 멤버
 - ▶ 변수 – 클래스 객체가 갖는 데이터
 - ▶ 메소드 – 클래스가 수행할 수 있는 일
- ▶ 객체화(instantiation)
 - ▶ `classType object = new classType(arguments);`

C# 키워드

예약어

abstract	as	base	bool	break	byte	case
catch	char	checked	class	const	continue	decimal
default	delegate	do	double	else	enum	event
explicit	extern	false	finally	fixed	float	for
foreach	goto	if	implicit	in	in (generic modifier)	int
interface	internal	is	lock	long	namespace	new
null	object	operator	out	out (generic modifier)	override	params
private	protected	public	readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc	static	string	struct
switch	this	throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using	virtual	void
volatile	while					

C# 키워드

- ▶ 문맥적 예약어(Contextual keywords)
 - ▶ 예약어는 아니지만, 코드에 특별한 의미를 부여한다.

add	alias	ascending	descending	dynamic	from	get
global	group	into	join	let	orderby	partial (type)
partial (method)	remove	select	set			



C# - Syntax

동명대학교 게임그래픽학과

강영민

자료형 (data types)

▶ 자료형의 종류

▶ 값(value type) – 기본 자료형

- ▶ bool, byte, char, decimal, double, float, int, long, sbyte, short, uint, ulong, ushort

▶ 참조(reference type) – 객체를 다루는 데에 사용

- ▶ 객체 – C# 공용 타입 시스템(CTS)에서 모든 자료형의 기본형, 컴파일 시간에 형 검사.
- ▶ 다이내믹 – 어떤 값 자료형이라도 동적인 자료형에 저장할 수 있으며, 이 동적 자료형은 실행 시간에 형 검사가 이루어진다.

- ▶ `Dynamic <variable_name> = value;`

- ▶ `Dynamic d = 20;`

- ▶ 스트링 – 객체 클래스에서 상속된 것.

▶ 포인터(pointer type)

- ▶ C, C++에서 사용하는 포인터를 원칙적으로 사용할 수 있다.
- ▶ unsafe 블록에서 컴파일 옵션을 /unsafe로 지정하고 사용 가능

형 변환(type conversion)

- ▶ 묵시적 형 변환(implicit type conversion)
 - ▶ C#에 의해 자동적으로 이루어지는 안전한 형 변환
 - ▶ 자바의 업캐스팅
- ▶ 명시적 형 변환(explicit type conversion)
 - ▶ 프로그래머가 명시적으로 수행하는 형 변환
 - ▶ C#이 책임지지 않는 형 변환
- ▶ C# 형 변환 메소드
 - ▶ ToBoolean, ToByte, ToChar, ToDateTime, ToDecimal, ToDouble, ToInt16, ToInt32, ToInt64, ToSbyte, ToSingle, ToString, ToType, ToUInt16, ToUInt32, ToUInt64

변수

- ▶ C, C++ 스타일 사용
- ▶ 변수를 입력 받는 예
 - ▶ `Int num;`
 - ▶ `Num = Convert.ToInt32(Console.ReadLine());`
- ▶ Lvalue와 Rvalue
 - ▶ Lvalue: 대입연산자(assignment)의 왼쪽과 오른쪽 모두에 나타날 수 있는 표현
 - ▶ 값을 저장할 수 있는 표현
 - ▶ Rvalue: 대입연산자의 오른쪽에만 나타날 수 있는 표현
 - ▶ 값을 저장할 수 없는 표현

상수 – 정수 리터럴

- ▶ 정수 리터럴
 - ▶ 212, 215u, 0xFeeL : 사용가능
 - ▶ 078, 032UU : 사용불가능
- ▶ 85: 10 진수(decimal)
- ▶ 0x4b: 16진수(hexadecimal)
- ▶ 30: int
- ▶ 30u: unsigned int
- ▶ 30l: long
- ▶ 30ul: unsigned long

상수 – 부동소수점 리터럴, 문자

▶ 부동소수점

- ▶ 3.14159, 31459E-5L : 사용가능
- ▶ 510E, 210F, .e55 : 사용불가능

▶ 문자

Escape sequence	Meaning
\\	\ character
\'	' character
\"	" character
\?	? character
\a	Alert or bell
\b	Backspace
\f	Form feed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\ooo	Octal number of one to three digits
\xhh . . .	Hexadecimal number of one or more digits

연산자

- ▶ C와 동일
 - ▶ C에 없는 연산자
 - ▶ `typeof()`
 - ▶ 클래스형을 리턴함
 - ▶ `is`
 - ▶ 클래스 객체가 특정한 형인지 확인
 - ▶ `if(Ford is Car)`
 - ▶ `as`
 - ▶ 형 변환이 실패해도 예외(exception)을 일으키지 않고 수행
- ```
Object obj = new String("Hello");
StringReader r = obj as StringReader;
```

## 연산자 우선순위

| Category       | Operator                          | Associativity |
|----------------|-----------------------------------|---------------|
| Postfix        | () [] -> . ++ --                  | Left to right |
| Unary          | + - ! ~ ++ -- (type)* & sizeof    | Right to left |
| Multiplicative | * / %                             | Left to right |
| Additive       | + -                               | Left to right |
| Shift          | << >>                             | Left to right |
| Relational     | < <= > >=                         | Left to right |
| Equality       | == !=                             | Left to right |
| Bitwise AND    | &                                 | Left to right |
| Bitwise XOR    | ^                                 | Left to right |
| Bitwise OR     |                                   | Left to right |
| Logical AND    | &&                                | Left to right |
| Logical OR     |                                   | Left to right |
| Conditional    | ?:                                | Right to left |
| Assignment     | = += -= *= /= %= >>= <<= &= ^=  = | Right to left |
| Comma          | ,                                 | Left to right |

# 의사결정 - 조건

- ▶ If 문
  - ▶ 조건에 맞으면 수행
- ▶ If-else 문
  - ▶ 조건에 맞는 경우와 그렇지 않은 경우 수행 조건 달리 설정
- ▶ 중첩된 if 문
  - ▶ 각각의 수행 조건이 만족할 경우 다시 조건 검사를 수행
- ▶ Switch 문
  - ▶ 변수가 가질 수 있는 값의 리스트와 대조하여 일치하는 값에 설정된 동작 수행
- ▶ 중첩된 switch
- ▶ condition?x:y;
  - ▶ Condition이 만족되면 x, 그렇지 않으면 y의 표현이 사용됨

# 반복

- ▶ 루프의 종류 – C와 차이 없음
  - ▶ While 루프
  - ▶ For 루프
  - ▶ Do... while 루프
  - ▶ 중첩된 루프
- ▶ 루프 제어 – C와 차이 없음
  - ▶ Break – 루프나 스위치를 벗어남
  - ▶ Continue – 반복 작업 단위의 나머지 작업을 종료하고 새롭게 반복작업을 수행



# C# - 객체지향

동명대학교 게임그래픽학과

강영민

# 캡슐화(encapsulation)

- ▶ 캡슐화(encapsulation)
  - ▶ 하나 이상의 개체를 물리적 혹은 논리적 패키지 안에 넣는 것
  - ▶ 객체지향 프로그래밍 입장에서의 의미
    - ▶ 구현 상세 (implementation details)에 대한 접근을 제한하는 것
- ▶ 연관된 개념
  - ▶ 추상화(abstraction)
    - ▶ 드러나야 할 개념만을 드러내는 것
    - ▶ 캡슐화는 이러한 추상화의 수준을 결정한다
- ▶ 캡슐화의 구현
  - ▶ 접근자(access specifier)를 통해 구현
  - ▶ public, private, protected, internal, protected internal

# 접근자

- ▶ private
  - ▶ 클래스의 멤버를 다른 함수나 객체가 접근할 수 없도록 숨기는 역할
- ▶ protected
  - ▶ 자식 클래스에서는 접근할 수 있도록 하지만, 다른 클래스에서는 접근이 불가능
- ▶ internal
  - ▶ 해당 클래스 멤버가 정의된 응용 프로그램 내에서는 어디서든 접근이 가능
    - = 현재의 어셈블리 내에 있는 다른 함수나 객체에 접근 권한 허용
- ▶ protected internal
  - ▶ 다른 클래스 객체나 함수들이 접근할 수 없지만, 해당 응용프로그램 내에 있거나 자식 클래스에서는 접근이 가능

# 메소드

- ▶ 메소드를 정의하는 방법

<접근자> <반환형> 메소드 이름 (파라미터 리스트)

{

    메소드 내용

}

# 메소드 사용 예

```
using System;
namespace CalculatorApplication
{
 class NumberManipulator
 {
 public int factorial(int num)
 {
 /* local variable declaration */
 int result;
 if (num == 1)
 {
 return 1;
 }
 else
 {
 result = factorial(num - 1) * num;
 return result;
 }
 }

 static void Main(string[] args)
 {
 NumberManipulator n = new NumberManipulator();
 //calling the factorial method
 Console.WriteLine("Factorial of 6 is : {0}", n.factorial(6));
 Console.WriteLine("Factorial of 7 is : {0}", n.factorial(7));
 Console.WriteLine("Factorial of 8 is : {0}", n.factorial(8));
 Console.ReadLine();
 }
 }
}
```



# 배열

- ▶ 선언 (배열 변수는 참조형 변수)
  - ▶ `datatype[] arrayName;`
- ▶ 초기화 - new가 필요
  - ▶ `double[] balance = new double[10];`
- ▶ 값 지정
  - ▶ 인덱스
    - ▶ `double[] balance = new double[10];`
    - ▶ `balance[0] = 4500.0;`
  - ▶ 선언시 지정
    - ▶ `double[] balance = { 2340.0, 4523.69, 3421.0};`
    - ▶ `int [] marks = new int[5] { 99, 98, 92, 97, 95};`
    - ▶ `int [] marks = new int[] { 99, 98, 92, 97, 95};`
- ▶ 복사 (참조만 복사)
  - ▶ `int [] marks = new int[] { 99, 98, 92, 97, 95}; int[] score = marks;`

# Foreach 루프

```
using System;
namespace ArrayApplication
{
 class MyArray
 {
 static void Main(string[] args)
 {
 int [] n = new int[10]; /* n is an array of 10 integers */

 /* initialize elements of array n */
 for (int i = 0; i < 10; i++)
 {
 n[i] = i + 100;
 }

 /* output each array element's value */
 foreach (int j in n)
 {
 int i = j-100;
 Console.WriteLine("Element[{0}] = {1}", i, j);
 i++;
 }
 Console.ReadKey();
 }
 }
}
```

Element[0] = 100  
Element[1] = 101  
Element[2] = 102  
Element[3] = 103  
Element[4] = 104  
Element[5] = 105  
Element[6] = 106  
Element[7] = 107  
Element[8] = 108  
Element[9] = 109

# C# 배열

- ▶ C#의 배열은 다음과 같은 개념을 지원한다

| 개념              | 내용                                                      |
|-----------------|---------------------------------------------------------|
| 다차원 배열          | C#은 다차원 배열을 지원한다                                        |
| 들쭉날쭉한 배열        | C#은 배열을 원소로 하는 다차원 배열을 지원한다                             |
| 배열을 함수에 인자로 넘기기 | 인덱스 없이 배열 이름만으로 배열에 대한 포인터를 함수에 넘길 수 있다                 |
| 파라미터 배열         | 함수에 정해지지 않은 수의 파라미터를 넘길때 배열을 사용할 수 있다                   |
| 배열 클래스          | System 네임스페이스에 배열 클래스가 정의되어 있고, 이 클래스가 모든 배열의 기본 클래스이다. |

# 스트링의 생성

## ▶ 스트링을 생성하는 방법

- ▶ 스트링 변수에 스트링 리터럴을 지정
- ▶ 스트링 클래스 생성자를 사용
- ▶ 스트링 잇기 연산자(+)를 사용
- ▶ 스트링을 반환하는 메소드를 호출하거나 프로퍼티를 얻어 오기
- ▶ 값이나 객체를 스트링 표현으로 바꾸는 포매팅 메소드를 호출하기

```
using System;
namespace StringApplication
{
 class Program
 {
 static void Main(string[] args)
 {
 //from string literal and string concatenation
 string fname, lname;
 fname = "Rowan";
 lname = "Atkinson";

 string fullname = fname + lname;
 Console.WriteLine("Full Name: {0}", fullname);

 //by using string constructor
 char[] letters = { 'H', 'e', 'l', 'l', 'o' };
 string greetings = new string(letters);
 Console.WriteLine("Greetings: {0}", greetings);

 //methods returning string
 string[] sarray = { "Hello", "From", "Tutorials", "Point" };
 string message = String.Join(" ", sarray);
 Console.WriteLine("Message: {0}", message);

 //formatting method to convert a value
 DateTime waiting = new DateTime(2012, 10, 10, 17, 58, 1);
 string chat = String.Format("Message sent at {0:t} on {0:D}", waiting);
 Console.WriteLine("Message: {0}", chat);
 }
 }
}
```

Full Name: Rowan Atkinson

Greetings: Hello

Message: Hello From Tutorials Point

Message: Message sent at 5:58 PM on Wednesday, October 10, 2012

# 스트링 클래스의 프로퍼티(Properties)

- ▶ Chars

- ▶ 현재 String 객체 내에서 특정한 위치에 있는 Char 객체를 얻음

- ▶ Length

- ▶ 현재 String 객체 내에 존재하는 문자의 수

# 스트링 클래스의 메소드

- public static int Compare(string strA, string strB)
- public static int Compare(string strA, string strB, bool ignoreCase)
- public static string Concat(string str0, string str1)
- public static string Concat(string str0, string str1, string str2)
- public static string Concat(string str0, string str1, string str2, string str3)
- public bool Contains(string value)
- public static string Copy (string str)
- public void CopyTo(int sourceIndex, char[] destination, int destinationIndex, int count)
- public bool EndsWith(string value)
- public bool Equals(string value)
- public static bool Equals(string a, string b)
- public static string Format(string format, Object arg0)
- public int IndexOf(char value)
- public int IndexOf(string value)
- public int IndexOf(char value, int startIndex)
- public int IndexOf(string value, int startIndex)
- public int IndexOfAny(char[] anyOf)
- public int IndexOfAny(char[] anyOf, int startIndex)
- public string Insert(int startIndex, string value)
- public static bool IsNullOrEmpty(string value)
- public static string Join(string separator, params string[] value)
- public static string Join(string separator, string[] value, int startIndex, int count)
- public int LastIndexOf(char value)
- public int LastIndexOf(string value)
- public string Remove(int startIndex)
- public string Remove(int startIndex, int count)
- public string Replace(char oldChar, char newChar)
- public string Replace(string oldValue, string newValue)
- public string[] Split(params char[] separator)
- public string[] Split(char[] separator, int count)
- public bool StartsWith(string value)
- public char[] ToCharArray()
- public char[] ToCharArray(int startIndex, int length)
- public string ToLower()
- public string ToUpper()
- public string Trim()

# 문자열 비교

```
using System;
namespace StringApplication
{
 class StringProg
 {
 static void Main(string[] args)
 {
 string str1 = "This is test";
 string str2 = "This is text";

 if (String.Compare(str1, str2) == 0)
 {
 Console.WriteLine(str1 + " and " + str2 + " are equal.");
 }
 else
 {
 Console.WriteLine(str1 + " and " + str2 + " are not equal.");
 }
 Console.ReadKey() ;
 }
 }
}
```

# 스트링 포함 검사

```
using System;
namespace StringApplication
{
 class StringProg
 {
 static void Main(string[] args)
 {
 string str = "This is test";
 if (str.Contains("test"))
 {
 Console.WriteLine("The sequence 'test' was found.");
 }
 Console.ReadKey();
 }
 }
}
```

This is test and This is text are not equal.

# 스트링 조인

```
using System;
namespace StringApplication
{
 class StringProg
 {
 static void Main(string[] args)
 {
 string[] starray = new string[]{"Down the way nights are dark",
 "And the sun shines daily on the mountain top",
 "I took a trip on a sailing ship",
 "And when I reached Jamaica",
 "I made a stop"};

 string str = String.Join("\n", starray);
 Console.WriteLine(str);
 }
 }
}
```

Down the way nights are dark  
And the sun shines daily on the mountain top  
I took a trip on a sailing ship  
And when I reached Jamaica  
I made a stop

# 서브스트링

```
using System;
namespace StringApplication
{
 class StringProg
 {
 static void Main(string[] args)
 {
 string str = "Last night I dreamt of San Pedro";
 Console.WriteLine(str);
 string substr = str.Substring(23);
 Console.WriteLine(substr);
 }
 }
}
```

San Pedro



# 구조체 (structure)

- ▶ C#의 구조체
  - ▶ 값 형태의 자료형
  - ▶ 하나의 변수가 다수의 자료형 변수를 포함할 수 있음
  - ▶ Struct 키워드 사용
- ▶ 사용자 정의 데이터의 생성
  - ▶ '책'이라는 자료형을 만들고 싶다
    - ▶ 책이라는 자료형이 담을 내용
      - ▶ 제목
      - ▶ 저자
      - ▶ 주제
      - ▶ 도서번호

```
struct Books
{
 public string title;
 public string author;
 public string subject;
 public int book_id;
};
```

# C# 구조체의 특징

- ▶ C/C++과 많이 다름
- ▶ C# 구조체의 특징
  - ▶ 메소드, 필드, 인덱서, 프로퍼티, 연산자 메소드, 이벤트를 가질 수 있다
  - ▶ 생성자는 가질 수 있지만, 소멸자는 가질 수 없다
    - ▶ 디폴트 생성자를 정의할 수는 없다. 디폴트 생성자는 자동으로 만들어진다.
  - ▶ 상속은 불가능
  - ▶ 하나 이상의 인터페이스 구현 가능
  - ▶ 구조체 멤버는 추상(abstract), 프로텍티드(protected), 가상(virtual)로 지정될 수 없다.
  - ▶ New를 이용하여 클래스처럼 생성할 수 있다. 적절한 생성자가 불린다.
  - ▶ New를 이용하지 않고도 생성할 수 있는데, 이때는 필드가 초기화되지 않고, 이 값들이 모두 정해지기 전까지는 사용할 수 없다.

# 클래스와 구조체

- ▶ 둘의 차이는
  - ▶ 참조 형과 값 형
  - ▶ 상속 가능 vs 상속 불가
  - ▶ 구조체는 디폴트 생성자를 가질 수 없다

# 클래스(Class)

- ▶ 클래스 정의
  - ▶ 객체의 청사진을 정의하는 것
  - ▶ 정의 방법

```
<access specifier> class class_name
{
 // member variables
 <access specifier> <data type> variable1;
 <access specifier> <data type> variable2;
 ...
 <access specifier> <data type> variableN;
 // member methods
 <access specifier> <return type> method1(parameter_list)
 {
 // method body
 }
 <access specifier> <return type> method2(parameter_list)
 {
 // method body
 }
 ...
 <access specifier> <return type> methodN(parameter_list)
 {
 // method body
 }
}
```

# 상속

## ▶ 상속의 개념

### ▶ Is-a 관계

▶ “Human” is a “Animal”

▶ “Asian” is a “Human”

### ▶ 상속의 방향

▶ 부모 → 자식

▶ Animal → Human → Asian

```
<access-specifier> class <base_class>
{
 ...
}
class <derived_class> : <base_class>
{
 ...
}
```

```
using System;
namespace InheritanceApplication
{
 class Shape
 {
 public void setWidth(int w)
 {
 width = w;
 }
 public void setHeight(int h)
 {
 height = h;
 }
 protected int width;
 protected int height;
 }

 // Derived class
 class Rectangle: Shape
 {
 public int getArea()
 {
 return (width * height);
 }
 }

 class RectangleTester
 {
 static void Main(string[] args)
 {
 Rectangle Rect = new Rectangle();

 Rect.setWidth(5);
 Rect.setHeight(7);

 // Print the area of the object.
 Console.WriteLine("Total area: {0}", Rect.getArea());
 Console.ReadKey();
 }
 }
}
```

# 슈퍼클래스 참조

- ▶ 슈퍼 클래스 참조 변수
  - ▶ base (자바에서는 super)

```
class Tabletop : Rectangle
{
 private double cost;
 public Tabletop(double l, double w) : base(l, w)
 { }
 public double GetCost()
 {
 double cost;
 cost = GetArea() * 70;
 return cost;
 }
 public void Display()
 {
 base.Display();
 Console.WriteLine("Cost: {0}", GetCost());
 }
}
```

# 다중 상속

- ▶ 지원되지 않는다
- ▶ 다중 상속의 개념을 구현하는 방법
  - ▶ 인터페이스 (Java와 동일)

```
class Shape
{
 public void setWidth(int w)
 {
 width = w;
 }
 public void setHeight(int h)
 {
 height = h;
 }
 protected int width;
 protected int height;
}
```

```
// Base class PaintCost
public interface PaintCost
{
 int getCost(int area);
}
```

```
// Derived class
class Rectangle : Shape, PaintCost
{
 public int getArea()
 {
 return (width * height);
 }
 public int getCost(int area)
 {
 return area * 70;
 }
}
```

# 다형성 (polymorphism)

## ▶ 정적 다형성

- ▶ 메소드 오버로딩
- ▶ 연산자 오버로딩

## ▶ 동적 다형성

### ▶ 추상 클래스

- ▶ 상속을 받은 클래스가 구체적인 동작을 결정
  - ▶ 추상 클래스는 생성 불가
  - ▶ Sealed로 선언된 클래스는 상속 불가
    - ▶ 추상 클래스는 sealed로 선언될 수 없음
    - ▶ 참조: 자바의 final

```
class Printdata
{
 void print(int i)
 {
 Console.WriteLine("Printing int: {0}", i);
 }

 void print(double f)
 {
 Console.WriteLine("Printing float: {0}" , f);
 }
}
```

```
abstract class Shape
{
 public abstract int area();
}
class Rectangle: Shape
{
 private int length;
 private int width;
 public Rectangle(int a=0, int b=0)
 {
 length = a;
 width = b;
 }
 public override int area ()
 {
 Console.WriteLine("Rectangle class area :");
 return (width * length);
 }
}
```



# 연산자 오버로딩

- ▶ 연산자 오버로딩
  - ▶ 빌트인 연산자에 대한 재정의 혹은 오버로딩
- ▶ 오버로딩 가능한 연산자

| Operators                             | Description                                                      |
|---------------------------------------|------------------------------------------------------------------|
| +, -, !, ~, ++, --                    | These unary operators take one operand and can be overloaded.    |
| +, -, *, /, %                         | These binary operators take one operand and can be overloaded.   |
| ==, !=, <, >, <=, >=                  | The comparison operators can be overloaded                       |
| &&,                                   | The conditional logical operators cannot be overloaded directly. |
| +=, -=, *=, /=, %=                    | The assignment operators cannot be overloaded.                   |
| =, ., ?:, ->, new, is, sizeof, typeof | These operators cannot be overloaded.                            |

```
class Box
{
 private double length; // Length of a box
 private double breadth; // Breadth of a box
 private double height; // Height of a box

 public double getVolume()
 {
 return length * breadth * height;
 }

 public void setLength(double len)
 {
 length = len;
 }

 public void setBreadth(double bre)
 {
 breadth = bre;
 }

 public void setHeight(double hei)
 {
 height = hei;
 }

 // Overload + operator to add two Box objects.
 public static Box operator+ (Box b, Box c)
 {
 Box box = new Box();
 box.length = b.length + c.length;
 box.breadth = b.breadth + c.breadth;
 box.height = b.height + c.height;
 return box;
 }
}
```

# 인터페이스

## ▶ 인터페이스

- ▶ 상속받는 모든 클래스가 지켜야 하는 구문적 약정
- ▶ 구현된 내용은 하나도 없음
  - ▶ 모든 메소드가 추상 메소드인 클래스
- ▶ 인터페이스는 다중 상속을 해도 문제가 발생하지 않음
  - ▶ 다중 상속을 위해서 interface를 활용

```
public interface ITransactions
{
 // interface members
 void showTransaction();
 double getAmount();
}
```